

ADVERTISEMENT

Python String Formatting

[< Previous](#)[Next >](#)

F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.

Before Python 3.6 we had to use the `format()` method.

F-Strings

F-string allows you to format selected parts of a string.

To specify a string as an f-string, simply put an `f` in front of the string literal, like this:

Example

[Get your own Python Server](#)

Create an f-string:

```
txt = f"The price is 49 dollars"
print(txt)
```

[Try it Yourself »](#)

Placeholders and Modifiers

To format values in an f-string, add placeholders `{}`, a placeholder can contain variables, operations, functions, and modifiers to format the value.

Example



COLOR
PICKER

Add a placeholder for the `price` variable:

```
price = 59
txt = f"The price is {price} dollars"
print(txt)
```

[Try it Yourself »](#)

A placeholder can also include a *modifier* to format the value.

A modifier is included by adding a colon `:` followed by a legal formatting type, like `.2f` which means fixed point number with 2 decimals:

Example

Display the price with 2 decimals:

```
price = 59
txt = f"The price is {price:.2f} dollars"
print(txt)
```

[Try it Yourself »](#)

You can also format a value directly without keeping it in a variable:

Example

Display the value `95` with 2 decimals:

```
txt = f"The price is {95:.2f} dollars"
print(txt)
```

[Try it Yourself »](#)

ADVERTISEMENT

[REMOVE ADS](#)



Perform Operations in F-Strings

You can perform Python operations inside the placeholders.

You can do math operations:

Example

Perform a math operation in the placeholder, and return the result:

```
txt = f"The price is {20 * 59} dollars"
print(txt)
```

Try it Yourself »

You can perform math operations on variables:

Example

Add taxes before displaying the price:

```
price = 59
tax = 0.25
txt = f"The price is {price + (price * tax)} dollars"
print(txt)
```

Try it Yourself »

You can perform `if...else` statements inside the placeholders:

Example

Return "Expensive" if the price is over 50, otherwise return "Cheap":

```
price = 49
txt = f"It is very {'Expensive' if price>50 else 'Cheap'}"
print(txt)
```

Try it Yourself »

Execute Functions in F-Strings

You can execute functions inside the placeholder:

Example

Use the string method `upper()` to convert a value into upper case letters:

```
fruit = "apples"
txt = f"I love {fruit.upper()}"
print(txt)
```

Try it Yourself »

The function does not have to be a built-in Python method, you can create your own functions and use them:

Example

Create a function that converts feet into meters:

```
def myconverter(x):
    return x * 0.3048

txt = f"The plane is flying at a {myconverter(30000)}
meter altitude"
print(txt)
```

Try it Yourself »

More Modifiers

At the beginning of this chapter we explained how to use the `.2f` modifier to format a number into a fixed point number with 2 decimals.

There are several other modifiers that can be used to format values:

Example

Use a comma as a thousand separator:

```
price = 59000
txt = f"The price is {price:,.} dollars"
print(txt)
```

Try it Yourself »

Here is a list of all the formatting types.

Formatting Types		
:<	Try it	Left aligns the result (within the available space)
:>	Try it	Right aligns the result (within the available space)
:^	Try it	Center aligns the result (within the available space)
:=	Try it	Places the sign to the left most position
:+	Try it	Use a plus sign to indicate if the result is positive or negative
:-	Try it	Use a minus sign for negative values only
:	Try it	Use a space to insert an extra space before positive numbers (and a minus sign before negative numbers)
:,	Try it	Use a comma as a thousand separator
:_	Try it	Use an underscore as a thousand separator
:b	Try it	Binary format
:c		Converts the value into the corresponding Unicode character
:d	Try it	Decimal format
:e	Try it	Scientific format, with a lower case e
:E	Try it	Scientific format, with an upper case E
:f	Try it	Fix point number format
:F	Try it	Fix point number format, in uppercase format (show <code>inf</code> and <code>nan</code> as <code>INF</code> and <code>NAN</code>)
:g		General format
:G		General format (using a upper case E for scientific notations)

<code>:o</code>	Try it	Octal format
<code>:x</code>	Try it	Hex format, lower case
<code>:X</code>	Try it	Hex format, upper case
<code>:n</code>		Number format
<code>:%</code>	Try it	Percentage format

String format()

Before Python 3.6 we used the `format()` method to format strings.

The `format()` method can still be used, but f-strings are faster and the preferred way to format strings.

The next examples in this page demonstrates how to format strings with the `format()` method.

The `format()` method also uses curly brackets as placeholders `{}`, but the syntax is slightly different:

Example

Add a placeholder where you want to display the price:

```
price = 49
txt = "The price is {} dollars"
print(txt.format(price))
```

Try it Yourself »

You can add parameters inside the curly brackets to specify how to convert the value:

Example

Format the price to be displayed as a number with two decimals:

```
txt = "The price is {:.2f} dollars"
```

Try it Yourself »

Check out all formatting types in our [String format\(\) Reference](#).

Multiple Values

If you want to use more values, just add more values to the `format()` method:

```
print(txt.format(price, itemno, count))
```

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Certificates ▼](#)[Sign In](#)[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C++](#) [C#](#) [BOO](#) [»](#)

[Python Range](#)
[Python Lambda](#)
[Python Arrays](#)
[Python OOP](#)
[Python Classes/Objects](#)
[Python Inheritance](#)
[Python Iterators](#)
[Python Polymorphism](#)
[Python Scope](#)
[Python Modules](#)
[Python Dates](#)
[Python Math](#)
[Python JSON](#)
[Python RegEx](#)
[Python PIP](#)
[Python Try...Except](#)
[Python String Formatting](#)
[Python None](#)
[Python User Input](#)

Example

```
quantity = 3
itemno = 567
price = 49
myorder = "I want {} pieces of item number {} for
 {:.2f} dollars."
print(myorder.format(quantity, itemno, price))
```

[Try it Yourself »](#)

Index Numbers

You can use index numbers (a number inside the curly brackets `{0}`) to be sure the values are placed in the correct placeholders:

Example

```
quantity = 3
itemno = 567
price = 49
myorder = "I want {0} pieces of item number {1} for
 {2:.2f} dollars."
print(myorder.format(quantity, itemno, price))
```

[Try it Yourself »](#)

Also, if you want to refer to the same value more than once, use the

AI

index number:

Example

```
age = 36
name = "John"
txt = "His name is {1}. {1} is {0} years old."
print(txt.format(age, name))
```

Try it Yourself »

Named Indexes

You can also use named indexes by entering a name inside the curly brackets `{carname}`, but then you must use names when you pass the parameter values `txt.format(carname = "Ford")`:

Example

```
myorder = "I have a {carname}, it is a {model}."
print(myorder.format(carname = "Ford", model = "Mustang"))
```

Try it Yourself »

Exercise ?

What placeholders are used when dealing with f-strings?

- ☐ []
- ☐ ()
- ☐ {}

Submit Answer »